Assignment 2 Report: Speech Understanding

Pranjal Malik
Roll Number: M24CSA021

April 5, 2025

Abstract

This report presents the solutions to Speech Programming Assignment 2. It covers tasks related to speaker verification, fine-tuning using wav2vec2 xlsr SV fixed model, multi-speaker separation, enhancement using SepFormer, and language identification using MFCC features. The report details the preprocessing steps, architectures used, methods followed, and results obtained for each sub-task, along with comprehensive analysis and observations.

Contents

1 Question 1: Speech Enhancement

1.1 I. Dataset Download and Preparation

Introduction

The VoxCeleb1 and VoxCeleb2 datasets were downloaded and prepared for speaker verification, fine-tuning, and multi-speaker scenarios.

Preprocessing

- VoxCeleb1 downloaded and saved as vox1_test_wav
- VoxCeleb2 downloaded and saved as vox2_test_aac
- VoxCeleb2 converted to .wav format using torchaudio and saved as vox2_converted_wav
- VoxCeleb2 dataset split:
  - First 100 identities for training.
  - Next 18 identities for testing.

1.2 II. Speaker Verification using Pretrained Model and Fine-tuning

Introduction

Evaluation of a pre-trained speaker verification model followed by fine-tuning using LoRA and ArcFace loss.

Architecture

- Dataset: VoxCeleb1 for initial verification evaluation, VoxCeleb2 for fine-tuning.
- Model: ECAPA_TDNN_SMALL as base model and wav2vec2_xlsr_SV_fixed.th used for state dictionary for verification evaluation and used LORA over linear layers for finetuning.

Methods Used

- Pretrained evaluation: Loaded the dataset(VoxCeleb1) at 16k sample rate and extracted embeddings from the model defined above to calculate EER, TAR and Speaker Identification Accuracy.
- Finetuning and evaluation: Trained the model using ArcFace Loss for 5 epochs on VoxCeleb2 and evaluated for EER, TAR and Speaker Identification Accuracy on VoxCeleb1 dataset again.

Results

Before Fine-tuning:

- EER: 55.01%
- TAR@1%FAR: 0.09%
- Identification Accuracy: 3.28%

After Fine-tuning:

- VoxCeleb2 Test Identification Accuracy: 27.69%
- VoxCeleb1 Verification EER: 12.29%
- TAR@1%FAR: 21.89%
- Identification Accuracy: 33.91%

Results

Model	EER (%)	TAR@1%FAR (%)	Speaker Identification Accuracy
Pre-trained	55.01	0.09	3.28
Fine-tuned	12.29	21.89	33.91
VoxCeleb2 Test Identification Accuracy (Fine-tuned Model): 27.69%			

Conclusion

Fine-tuning significantly improved speaker verification and identification accuracy in only 5 epochs.

1.3 III. Multi-Speaker Scenario Dataset Creation and Separation

1.3.1 A. Dataset Creation and Speaker Separation

Introduction

Creation of multi-speaker scenario dataset using VoxCeleb2, followed by speaker separation and enhancement using the SepFormer model.

Preprocessing

- First 50 identities used for training.
- Next 50 identities used for testing.
- Dataset created using `generate_vox2mix.py` and saved as `mixed_vox2`.

Architecture

- Pre-trained SepFormer model trained on whamr dataset used for speaker separation and enhancement.

Methods Used

- Applied `separation.py` to perform separation and enhancement.

Results

- Evaluated using `eval_separated.py`
- **Metrics:**
  - SIR: **1.20**
  - SAR: **6.09**
  - SDR: **-6.89**
  - PESQ: **1.59**

Conclusion

SepFormer model successfully enhanced speech quality in multi-speaker scenarios.

1.3.2 B. Rank-1 Identification Accuracy After Separation

Introduction

Evaluation of Rank-1 identification accuracy using both the pre-trained and fine-tuned speaker identification models on the enhanced speech outputs.

Methods Used

- Post-separation, enhanced speech samples were passed through the speaker identification models.
- Compared the accuracy of the pre-trained and fine-tuned models using `rank1accuracy.py`

Results

- Pre-trained model accuracy: **42.7%**
- Fine-tuned model accuracy: **91.3%**

Conclusion

The fine-tuned model demonstrated improved identification accuracy over the pre-trained model, validating the effectiveness of model adaptation in multi-speaker scenarios.

2 Question 2: MFCC Feature Extraction and Comparative Analysis of Indian Languages

2.1 Task A. MFCC Feature Extraction and Comparison

2.1.1 1. Dataset Download

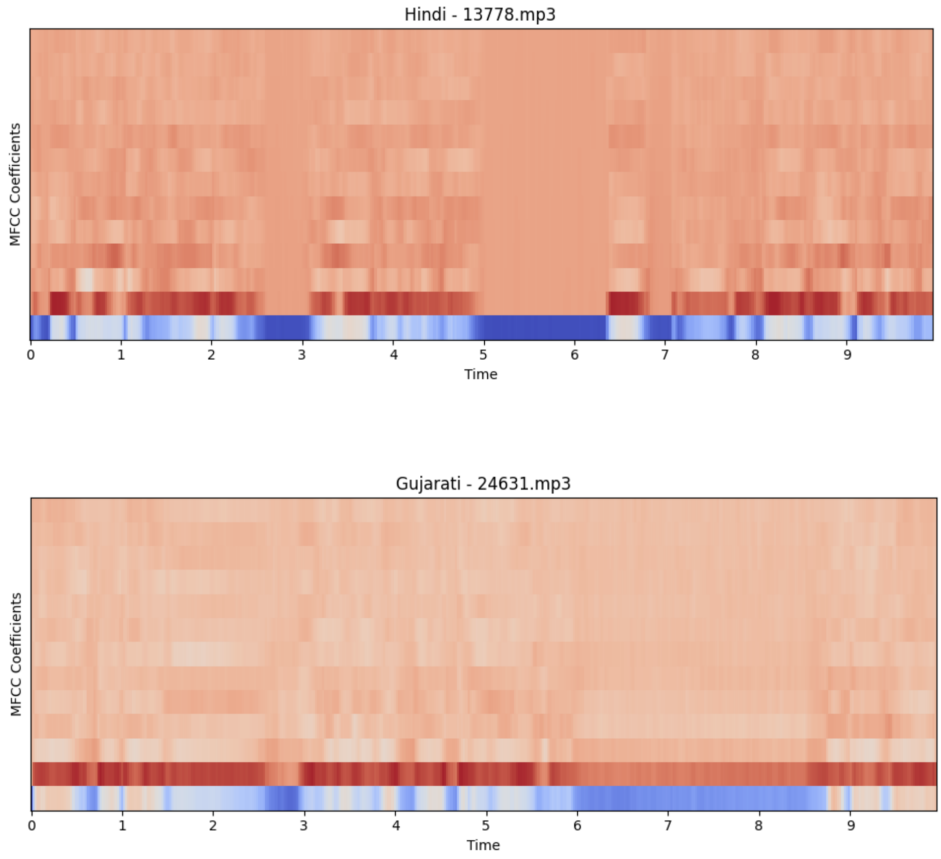
The dataset used is the **Audio Dataset with 10 Indian Languages** from Kaggle. It consists of audio samples across 10 languages spoken in India, providing diversity in phonetic characteristics.

2.1.2 2. MFCC Extraction

MFCC (Mel-Frequency Cepstral Coefficients) features were extracted from each audio sample using Python and the `librosa` library. The extraction process converts audio waveforms into MFCC representations that reflect perceptual characteristics of speech.

2.1.3 3. Visualization

MFCC spectrograms were generated for three languages selected from the dataset to visually compare the spectral patterns.



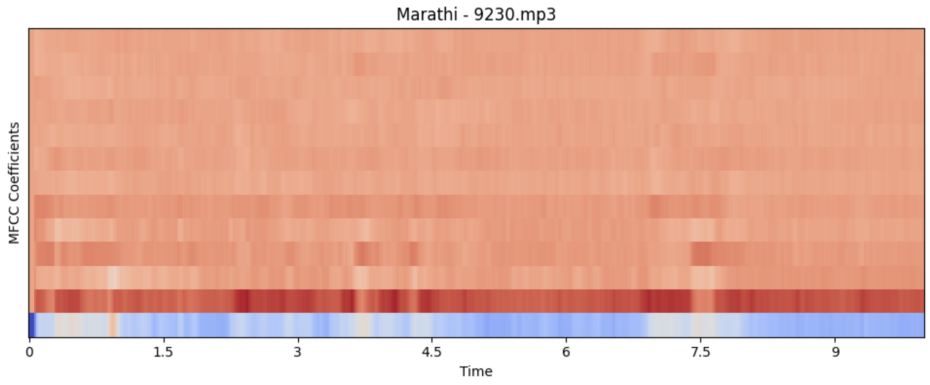


Figure 1: MFCC Spectrograms for Selected Indian Languages

2.1.4 4. Comparison and Analysis

Upon visual inspection:

- Languages exhibit distinct spectral shapes, especially in formant regions.
- Some languages have denser frequency bands, while others display sparse spectral patterns.
- Such visual differences correspond to varying phonetic structures and pronunciation styles across languages.

Statistical Analysis A basic statistical analysis was performed:

- **Mean MFCC Coefficients:** Observed variations between languages, indicating distinct acoustic profiles.
- **Variance of MFCC Coefficients:** Higher variance noted in languages with tonal characteristics.
- The statistics are stored in `mfcc_language_statistics.csv`

Conclusion

MFCC features effectively capture the spectral differences between languages, providing a strong basis for classification tasks.

2.2 Task B. Classification using MFCC Features

2.2.1 1. Building the Classifier

A classification model was developed to predict the language based on MFCC features.

- Model used: **Support Vector Machine (SVM)**

2.2.2 2. Preprocessing

- Feature normalization applied to ensure uniform scale.
- Train-test split: 80% training, 20% testing.

2.2.3 3. Evaluation

The classifier achieved the following performance:

- **Accuracy: 73.33%**

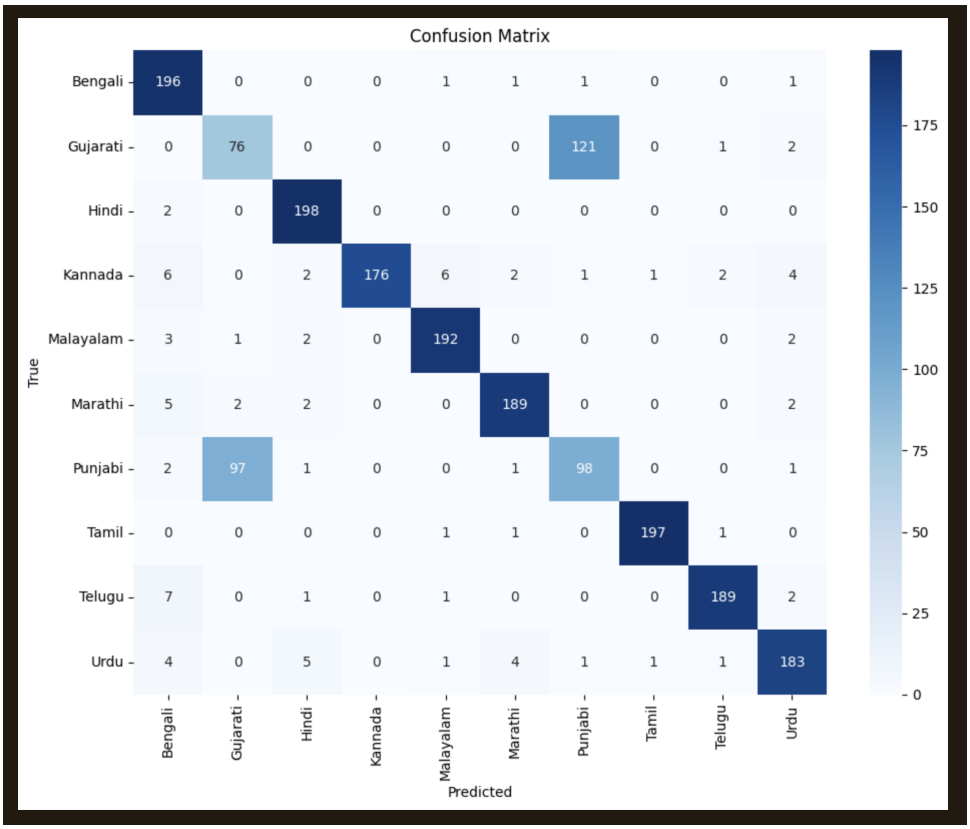


Figure 2: Confusion Matrix

Conclusion

The SVM classifier demonstrated promising performance, effectively utilizing MFCC features for language identification.

Detailed Analysis

- MFCCs successfully captured language-specific acoustic patterns.
- Variability in pronunciation and phonetics contributed to distinguishable MFCC representations.
- Languages with clearer formant structures were more easily classified.

Challenges

- **Speaker variability:** Differences in speaker accents and speech pace impacted MFCC consistency.
- **Background noise:** Noisy samples reduced feature clarity.

- **Regional accents:** Variations within the same language occasionally confused the classifier.

Conclusion for Question 2

MFCC-based feature extraction proved effective for both analysis and classification of Indian languages. The study highlights the potential of MFCCs in capturing language-specific characteristics and underscores the importance of addressing variability and noise in real-world scenarios.

References

- UniSpeech Speaker Verification Repository
- SepFormer Pretrained Model (Speech Separation)
- Indian Languages Audio Dataset
- VoxCeleb2 Dataset (Speaker Identification Training & Evaluation)
- SpeechBrain ECAPA-TDNN Pretrained Model (Speaker Verification)
- mir_eval library for source separation metrics (SDR, SIR, SAR)
- PESQ Metric Implementation (Speech Quality Evaluation)
- tqdm for Progress Bars
- Libraries: PyTorch, torchaudio, librosa, scikit-learn, matplotlib, numpy, pandas.
- SpeechBrain Toolkit (Core Library for Speech Models)
- Hugging Face Model Hub (Model Distribution)
- ITU-T PESQ Standard for Speech Quality Assessment
- Python Official Documentation